

HDOC DAY – 27

ALGORITHM:

Main Program Algorithm

1. Start.
2. Declare choice.
3. Display the menu:
 - Full Pyramid
 - Inverted Pyramid
 - Diamond Pattern
 - Palindromic Number Pyramid
 - Repeated-Number Pyramid
 - Exit
4. Input the user's choice.
5. Use a switch statement.
6. According to the choice, call the corresponding function.
7. If the choice is invalid, display "Invalid choice!".
8. Repeat the menu until the user selects Exit.
9. Stop.

Activity 1: Full Pyramid

1. Input number of rows n.
2. Check that n is positive.
3. For i = 1 to n:
 - Print n-i spaces.
 - Print 2*i-1 asterisks.
 - Move to the next line.
4. Return to the menu.

Activity 2: Inverted Pyramid

1. Input number of rows n .
2. Check that n is positive.
3. For $i = n$ down to 1:
 - Print $n-i$ spaces.
 - Print $2*i-1$ asterisks.
 - Move to the next line.
4. Return to the menu.

Activity 3: Diamond Pattern

1. Input n .
2. Check that n is positive.
3. Print the upper half:
 - For $i = 1$ to n , print $n-i$ spaces.
 - Print $2*i-1$ asterisks.
4. Print the lower half:
 - For $i = n-1$ down to 1, print $n-i$ spaces.
 - Print $2*i-1$ asterisks.
5. Return to the menu.

Activity 4: Palindromic Number Pyramid

1. Input n .
2. Check that n is positive.
3. For each row i from 1 to n :
 - Print spaces for alignment.
 - Print numbers from 1 to i .
 - Print numbers from $i-1$ down to 1.
4. Move to the next line after every row.
5. Return to the menu.

Activity 5: Repeated-Number Pyramid

1. Input n .

2. Check that n is positive.
3. For every row i from 1 to n:
 - Print spaces for alignment.
 - Print the value i, $2*i-1$ times.
4. Move to the next line.
5. Return to the menu.

C PROGRAM:

```
#include <stdio.h>

/* Function declarations */
void fullPyramid(void);
void invertedPyramid(void);
void diamondPattern(void);
void palindromicPyramid(void);
void repeatedNumberPyramid(void);

int main()
{
    int choice;

    do
    {
        printf("\n--- Pyramid/Diamond Pattern Menu ---\n");
        printf("1. Full Pyramid\n");
        printf("2. Inverted Pyramid\n");
        printf("3. Diamond Pattern\n");
        printf("4. Palindromic Number Pyramid\n");
        printf("5. Repeated-Number Pyramid\n");
```

```
printf("6. Exit\n");
```

```
printf("Enter your choice: ");
```

```
scanf("%d", &choice);
```

```
switch (choice)
```

```
{
```

```
    case 1:
```

```
        fullPyramid();
```

```
        break;
```

```
    case 2:
```

```
        invertedPyramid();
```

```
        break;
```

```
    case 3:
```

```
        diamondPattern();
```

```
        break;
```

```
    case 4:
```

```
        palindromicPyramid();
```

```
        break;
```

```
    case 5:
```

```
        repeatedNumberPyramid();
```

```
        break;
```

```
    case 6:
```

```

        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice!\n");
    }

} while (choice != 6);

return 0;
}

/* 1. Full Pyramid */
void fullPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {

```

```
    /* Print spaces */  
    for (j = 1; j <= n - i; j++)  
    {  
        printf(" ");  
    }  
  
    /* Print stars */  
    for (j = 1; j <= 2 * i - 1; j++)  
    {  
        printf("*");  
    }  
  
    printf("\n");  
}  
}
```

```
/* 2. Inverted Pyramid */  
void invertedPyramid(void)  
{  
    int n, i, j;  
  
    printf("Enter the number of rows: ");  
    scanf("%d", &n);  
  
    if (n <= 0)  
    {  
        printf("Number of rows must be positive.\n");  
        return;  
    }  
}
```

```

    }

    for (i = n; i >= 1; i--)
    {
        /* Print spaces */
        for (j = 1; j <= n - i; j++)
        {
            printf(" ");
        }

        /* Print stars */
        for (j = 1; j <= 2 * i - 1; j++)
        {
            printf("*");
        }

        printf("\n");
    }
}

```

```

/* 3. Diamond Pattern */
void diamondPattern(void)
{
    int n, i, j;

    printf("Enter the number of rows for upper half: ");
    scanf("%d", &n);

```

```
if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}
```

```
/* Upper half of diamond */
```

```
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= n - i; j++)
    {
        printf(" ");
    }

    for (j = 1; j <= 2 * i - 1; j++)
    {
        printf("*");
    }

    printf("\n");
}
```

```
/* Lower half of diamond */
```

```
for (i = n - 1; i >= 1; i--)
{
    for (j = 1; j <= n - i; j++)
    {
        printf(" ");
    }
```

```

    }

    for (j = 1; j <= 2 * i - 1; j++)
    {
        printf("*");
    }

    printf("\n");
}

}

/* 4. Palindromic Number Pyramid */
void palindromicPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        /* Print spaces */

```

```

        for (j = 1; j <= n - i; j++)
        {
            printf(" ");
        }

        /* Increasing numbers */
        for (j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }

        /* Decreasing numbers */
        for (j = i - 1; j >= 1; j--)
        {
            printf("%d ", j);
        }

        printf("\n");
    }
}

/* 5. Repeated-Number Pyramid */
void repeatedNumberPyramid(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

```

```

if (n <= 0)
{
    printf("Number of rows must be positive.\n");
    return;
}

for (i = 1; i <= n; i++)
{
    /* Print spaces */
    for (j = 1; j <= n - i; j++)
    {
        printf(" ");
    }

    /* Print row number 2*i-1 times */
    for (j = 1; j <= 2 * i - 1; j++)
    {
        printf("%d ", i);
    }

    printf("\n");
}
}

```

TEST CASE TABLE:

TEST CASE	CHOICE	INPUT(n)	EXPECTED RESULT
1	1	4	Full pyramid of 4 rows

2	2	4	Inverted pyramid of 4 rows
3	3	4	Diamond containing 7 rows
4	4	4	Palindromic number pyramid of 4 rows
5	5	4	Repeated-number pyramid of 4 rows
6	1	1	Prints one *
7	3	1	Prints one *
8	4	0	Displays Number of rows must be positive.
9	5	-2	Displays Number of rows must be positive.
10	7	--	Displays Invalid choice!
11	6	--	Exits the program

— Pyramid/Diamond Pattern Menu —

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 1

Enter the number of rows: 4

```
*
**
***
****
*****
*****
```

— Pyramid/Diamond Pattern Menu —

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 2

Enter the number of rows: 4

```
*****
****
***
**
*
```

— Pyramid/Diamond Pattern Menu —

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern
4. Palindromic Number Pyramid
5. Repeated-Number Pyramid
6. Exit

Enter your choice: 3

Enter the number of rows for upper half: 4

```
*
**
***
****
*****
*****
****
***
**
*
```

— Pyramid/Diamond Pattern Menu —

1. Full Pyramid
2. Inverted Pyramid
3. Diamond Pattern